

TLS, HTTP/2 & QUIC Security Lab

Handshake Analysis, Certificate Trust and
TCP Reset Testing

Valentin Ochioiu
Network Security Student

Folkuniversitetet, Gothenburg
May 2026

Project Overview

This lab analyses how encrypted web protocols establish trust and session keys, how HTTP/2 and HTTP/3 negotiate transport, and how TCP-based sessions react to forged reset packets.

Protocol analysis

- ▶ Build a TLS 1.2 HTTPS service with Nginx and OpenSSL.
- ▶ Inspect ClientHello, ServerHello, certificate and ECDHE exchange in Wireshark.
- ▶ Enable HTTP/2 through ALPN and compare the TLS 1.3 handshake.
- ▶ Build HTTP/3 with Caddy and inspect QUIC Initial and Handshake packets.

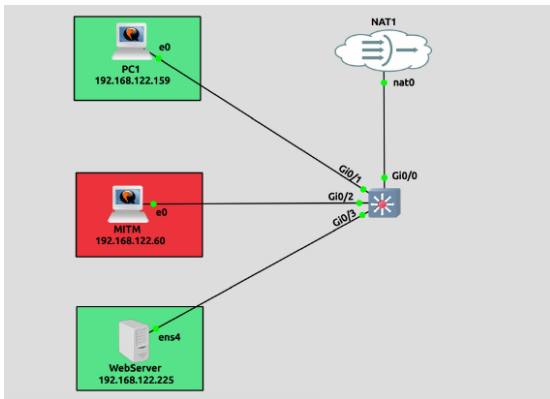
Security focus

Certificate trust, Perfect Forward Secrecy, protected application data and transport-layer disruption.

Validation objective

Compare what changes between TCP/TLS, HTTP/2 and QUIC while keeping the observations grounded in packet captures.

Test Model



GNS3 test topology: client 192.168.122.159, MITM node 192.168.122.60 and web server 192.168.122.225.

Services and tools

- ▶ Nginx + OpenSSL for HTTPS and TLS 1.2.
- ▶ Caddy v2 for HTTP/3 over QUIC.
- ▶ Wireshark for handshake and packet analysis.
- ▶ curl for HTTP/1.1, HTTP/2 and HTTP/3 validation.
- ▶ tcpkill, hping3 and Scapy for TCP reset experiments.
- ▶ ARP spoofing and tcpdump/tshark for the MITM phase.

Scope

The lab is isolated and uses designated test hosts only.

HTTPS Server and Self-Signed Certificate

2.1 - Nginx & OpenSSL

```
debian@server:~$ sudo apt install -y nginx openssl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nginx is already the newest version (1.22.1-9+deb12u7).
openssl is already the newest version (3.0.20-1-deb12u1).
```

2.2 - Självsignerat certifikat

[illegible]

OpenSSL certificate generation with an RSA 4096-bit private key and SHA-256 signature.

```
GNU nano 7.2 /etc/nginx/sites-available/default *
# Virtual Host configuration for example.com
#
# You can move that to a different file under sites-available/ and symlink that
# to sites-enabled/ to enable it.
#
server {
    listen 443 ssl;
    server_name demo.local;

    ssl_certificate /etc/ssl/certs/selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/selfsigned.key;

    ssl_protocols TLSv1.2;
    ssl_ciphers HIGH:!aNULL:!MD5;

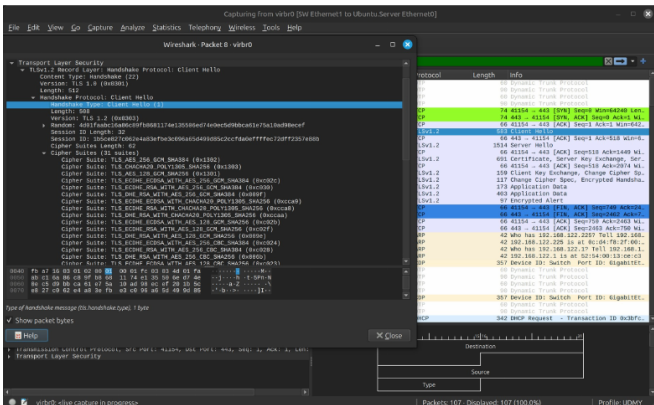
    root /var/www/html;
}
```

Nginx configured for HTTPS on port 443 with TLS 1.2.

Trust model

The certificate follows X.509 structure, but Subject and Issuer are the same because it is self-signed. Browsers therefore cannot build a chain to a trusted CA.

TLS 1.2: ClientHello



Wireshark view of the ClientHello and the advertised cipher suites/extensions.

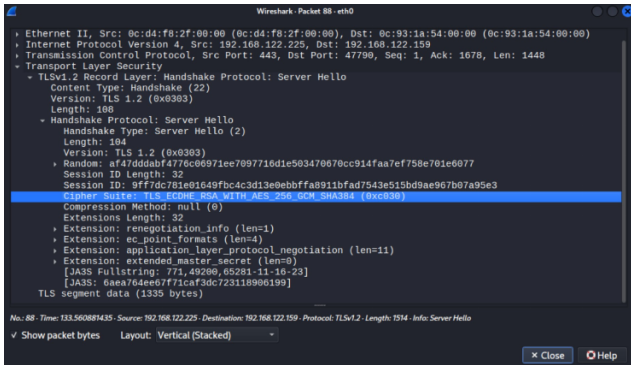
What the client advertises

- ▶ Supported TLS versions.
- ▶ Cipher suites and cryptographic capabilities.
- ▶ Extensions used to negotiate session features.
- ▶ A new TLS session is created for each independent curl connection.

Observed result

The server later selected TLS 1.2 because of the Nginx configuration.

TLS 1.2: Selected Cipher Suite

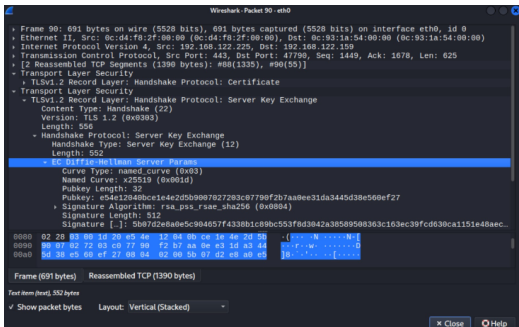


ServerHello selecting TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384.

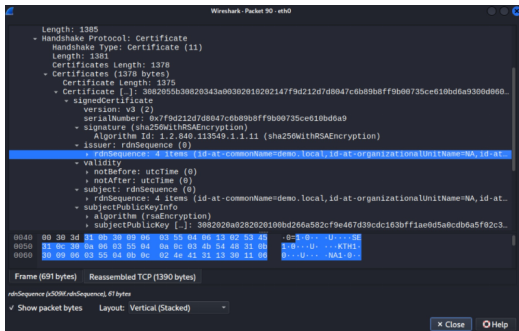
Cipher components

- ▶ **ECDHE** — ephemeral key agreement.
- ▶ **RSA** — server authentication through the certificate.
- ▶ **AES-256-GCM** — authenticated encryption for application traffic.
- ▶ **SHA-384** — hash function used by the suite and handshake construction.

ECDHE and Certificate Identity



Server Key Exchange: ECDHE using the X25519 elliptic curve.

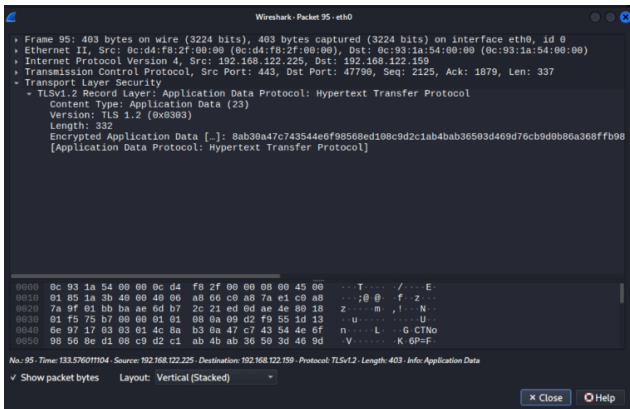


Certificate details show the same identity in Issuer and Subject for demo.local.

Security property

Ephemeral ECDHE creates temporary session keys and supports Perfect Forward Secrecy: compromise of the long-term RSA key does not automatically reveal previously captured session keys.

TLS 1.2: Handshake Completion



After the Finished messages, Wireshark shows encrypted TLS Application Data.

What this confirms

- ▶ Client and server completed the handshake.
- ▶ Shared traffic keys were established.
- ▶ HTTP content moved inside encrypted TLS records.
- ▶ The self-signed certificate still produced a trust warning at the client.

HTTP/2 Negotiation with ALPN

```
WebbServer
GNU nano 7.2 /etc/nginx/sites-available/default *

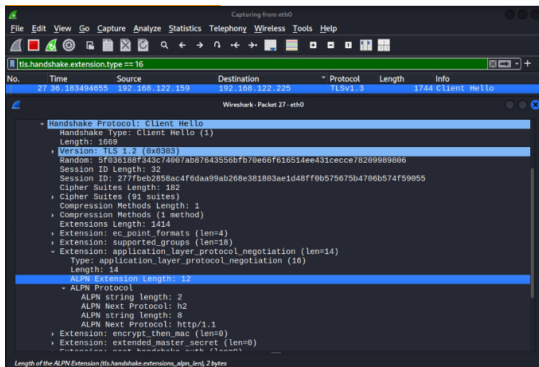
# Virtual Host configuration for example.com
#
# You can move that to a different file under sites-available/ and symlink that
# to sites-enabled/ to enable it.
#
server {
    listen 443 ssl http2;
    server_name demo.local;

    ssl_certificate /etc/ssl/certs/selfsigned.crt;
    ssl_certificate_key /etc/ssl/private/selfsigned.key;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    root /var/www/html;
}
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^I Execute  ^C Location
^X Exit      ^R Read File ^N Replace   ^U Paste     ^J Justify  ^_ Go To Line
```

Nginx enabled HTTP/2 and allowed TLS 1.2 / TLS 1.3.



ClientHello ALPN extension advertises h2 and http/1.1.

Key observation

HTTP/2 is negotiated inside the TLS handshake through ALPN. The capture used TLS 1.3, so more of the handshake became encrypted earlier than in the TLS 1.2 capture.

TLS 1.2, HTTP/2 and QUIC: Transport Comparison

Protocol	Transport	Handshake behaviour	Lab observation
TLS 1.2	TCP + TLS	TCP handshake, then a multi-flight TLS handshake	More visible handshake messages before Application Data
HTTP/2	TCP + TLS 1.3	TCP remains; ALPN selects h2 inside TLS	Earlier encrypted handshake traffic and fewer visible TLS messages
HTTP/3 / QUIC	UDP + QUIC + integrated TLS 1.3	Transport and cryptographic setup are combined	QUIC Initial / Handshake packets; first connection approximately 1-RTT

Important distinction

HTTP/2 improves multiplexing but still relies on TCP. QUIC removes TCP and integrates TLS 1.3 directly into the transport protocol.

HTTP/3 Server with Caddy v2

```
WebbServer
GNU nano 7.2 /etc/caddy/Caddyfile
# The Caddyfile is an easy way to configure your Caddy web server.
#
# Unless the file starts with a global options block, the first
# uncommented line is always the address of your site.
#
# To use your own domain name (with automatic HTTPS), first make
# sure your domain's A/AAAA DNS records are properly pointed to
# this machine's public IP, then replace ":80" below with your
# domain name.

:443 {
    tls /etc/caddy/selfsigned.crt /etc/caddy/selfsigned.key
    encode gzip
    respond "Hello QUIC!"
}
```

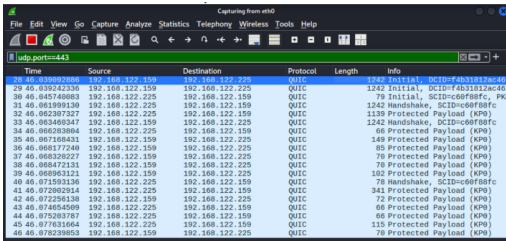
Caddy configured for port 443 with the lab certificate and a simple HTTP/3 response.

Permission problem Caddy initially could not read the private RSA key.

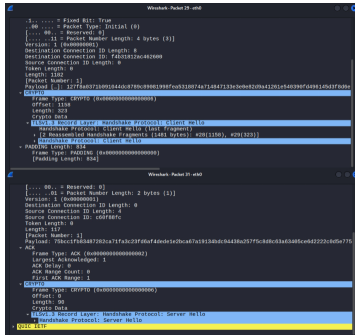
- ▶ Copied the certificate and key to /etc/caddy/.
- ▶ Changed ownership to the caddy service account.
- ▶ Restricted the private key with `chmod 600`.
- ▶ Verified the service was active after the fix.

```
debian@server:~$ sudo systemctl status caddy
● caddy.service - Caddy
   Loaded: loaded (/lib/systemd/system/caddy.service; enabled; preset: enabled)
   Active: active (running) since Wed 2026-05-27 13:11:36 UTC; 11min ago
     Docs: https://caddyserver.com/docs/
   Main PID: 2002 (caddy)
    Tasks: 8 (limit: 5283)
   Memory: 18.5M
      CPU: 67ms
   CGroup: /system.slice/caddy.service
           └─2002 /usr/bin/caddy run --environ --config /etc/caddy/Caddyfile
```

QUIC and HTTP/3 Handshake



Time	Source	Destination	Protocol	Length	Info
28.46.839692886	192.168.122.159	192.168.122.225	QUIC	1242	Initial, DCID=f4b31812ac40...
29.46.039242336	192.168.122.159	192.168.122.225	QUIC	78	Initial, DCID=f4b31812ac40...
30.46.045746083	192.168.122.225	192.168.122.159	QUIC	1242	Handshake, SCID=c60f88fc, PK...
31.46.061999130	192.168.122.225	192.168.122.159	QUIC	1139	Protected Payload (KP0)
32.46.062307327	192.168.122.225	192.168.122.159	QUIC	1242	Handshake, SCID=c60f88fc
33.46.063406347	192.168.122.159	192.168.122.225	QUIC	1242	Handshake, DCID=c60f88fc
34.46.066283804	192.168.122.225	192.168.122.159	QUIC	66	Protected Payload (KP0)
35.46.067108433	192.168.122.159	192.168.122.225	QUIC	149	Protected Payload (KP0)
36.46.068177248	192.168.122.159	192.168.122.225	QUIC	85	Protected Payload (KP0)
37.46.068326227	192.168.122.159	192.168.122.225	QUIC	78	Protected Payload (KP0)
38.46.068472131	192.168.122.159	192.168.122.225	QUIC	78	Protected Payload (KP0)
39.46.068963121	192.168.122.159	192.168.122.225	QUIC	102	Protected Payload (KP0)
40.46.071593136	192.168.122.225	192.168.122.159	QUIC	78	Handshake, SCID=c60f88fc
41.46.072002914	192.168.122.225	192.168.122.159	QUIC	341	Protected Payload (KP0)
42.46.072256138	192.168.122.159	192.168.122.225	QUIC	72	Protected Payload (KP0)
43.46.074654509	192.168.122.225	192.168.122.159	QUIC	66	Protected Payload (KP0)
44.46.075203787	192.168.122.225	192.168.122.159	QUIC	66	Protected Payload (KP0)
45.46.077631664	192.168.122.225	192.168.122.159	QUIC	115	Protected Payload (KP0)
46.46.078239853	192.168.122.159	192.168.122.225	QUIC	78	Protected Payload (KP0)



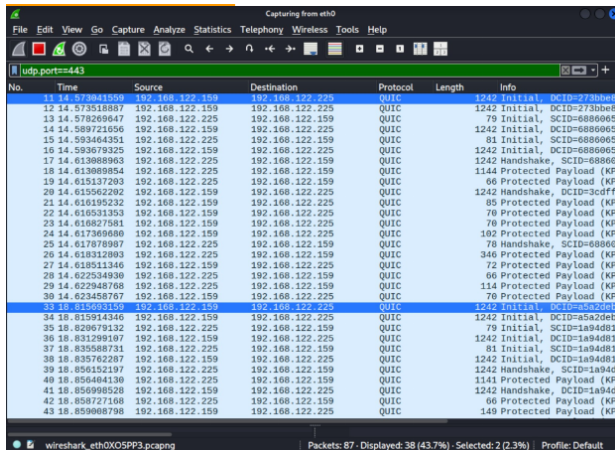
QUIC Initial, Handshake and Protected Payload packets over UDP/443.

TLS 1.3 ClientHello and ServerHello carried inside QUIC CRYPTO frames.

Key difference

TLS 1.3 is part of QUIC instead of running as a separate layer above TCP. This reduces connection setup latency and avoids TCP head-of-line blocking between independent QUIC streams.

0-RTT vs 1-RTT: What the Capture Showed



No.	Time	Source	Destination	Protocol	Length	Info
11	14.573941559	192.168.122.159	192.168.122.225	QUIC	1242	Initial, DCID=273bbe8
12	14.573518887	192.168.122.159	192.168.122.225	QUIC	1242	Initial, DCID=273bbe8
13	14.578269647	192.168.122.225	192.168.122.159	QUIC	79	Initial, SCID=6886065
14	14.589721656	192.168.122.159	192.168.122.225	QUIC	1242	Initial, DCID=6886065
15	14.593464351	192.168.122.225	192.168.122.159	QUIC	81	Initial, SCID=6886065
16	14.593679325	192.168.122.159	192.168.122.225	QUIC	1242	Initial, DCID=6886065
17	14.613088963	192.168.122.225	192.168.122.159	QUIC	1242	Handshake, SCID=6886065
18	14.613088954	192.168.122.225	192.168.122.159	QUIC	1144	Protected Payload (KP)
19	14.615137203	192.168.122.225	192.168.122.159	QUIC	66	Protected Payload (KP)
20	14.615562202	192.168.122.159	192.168.122.225	QUIC	1242	Handshake, DCID=3cdf
21	14.616195232	192.168.122.159	192.168.122.225	QUIC	85	Protected Payload (KP)
22	14.616531353	192.168.122.159	192.168.122.225	QUIC	70	Protected Payload (KP)
23	14.616827581	192.168.122.159	192.168.122.225	QUIC	70	Protected Payload (KP)
24	14.617369680	192.168.122.225	192.168.122.225	QUIC	102	Protected Payload (KP)
25	14.617878987	192.168.122.225	192.168.122.159	QUIC	78	Handshake, SCID=6886065
26	14.618312003	192.168.122.225	192.168.122.159	QUIC	346	Protected Payload (KP)
27	14.618511346	192.168.122.159	192.168.122.225	QUIC	72	Protected Payload (KP)
28	14.622534930	192.168.122.225	192.168.122.159	QUIC	66	Protected Payload (KP)
29	14.622948768	192.168.122.225	192.168.122.159	QUIC	114	Protected Payload (KP)
30	14.623458767	192.168.122.159	192.168.122.225	QUIC	70	Protected Payload (KP)
33	18.815603159	192.168.122.159	192.168.122.225	QUIC	1242	Initial, DCID=a5a2deb
34	18.815914346	192.168.122.159	192.168.122.225	QUIC	1242	Initial, DCID=a5a2deb
35	18.820679132	192.168.122.225	192.168.122.159	QUIC	79	Initial, SCID=1a94d81
36	18.831299107	192.168.122.159	192.168.122.225	QUIC	1242	Initial, DCID=1a94d81
37	18.835588731	192.168.122.225	192.168.122.159	QUIC	81	Initial, SCID=1a94d81
38	18.835762287	192.168.122.159	192.168.122.225	QUIC	1242	Initial, DCID=1a94d81
39	18.856152197	192.168.122.225	192.168.122.159	QUIC	1242	Handshake, SCID=1a94d81
40	18.856404130	192.168.122.225	192.168.122.159	QUIC	1141	Protected Payload (KP)
41	18.856998528	192.168.122.159	192.168.122.225	QUIC	1242	Handshake, DCID=1a94d81
42	18.858727168	192.168.122.225	192.168.122.159	QUIC	66	Protected Payload (KP)
43	18.859008798	192.168.122.159	192.168.122.225	QUIC	149	Protected Payload (KP)

Two QUIC connections showed almost the same Initial / Handshake / Protected Payload pattern.



```
kali@PCH:~$ curl -k --http3 -o /dev/null -s -w "time_starttransfer:%[time_starttransfer] \n time_total:%[time_total] \n" https://192.168.122.225
time_starttransfer:0.039427
time_total:0.039427

kali@PCH:~$ curl -k --http3 -o /dev/null -s -w "time_starttransfer:%[time_starttransfer] \n time_total:%[time_total] \n" https://192.168.122.225
time_starttransfer:0.032242
time_total:0.032329

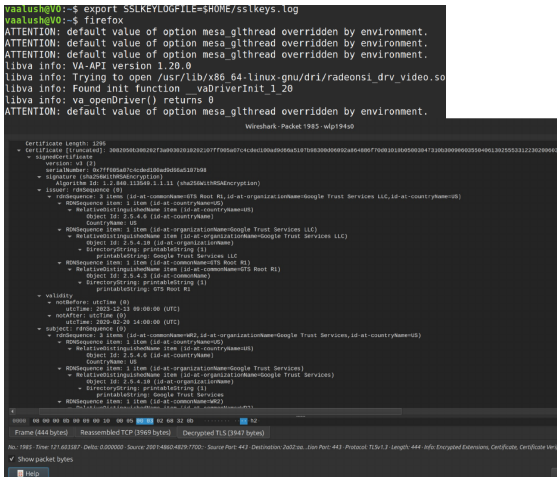
kali@PCH:~$
```

The second connection was slightly faster in curl timing.

Evidence limit

No clear application data was observed before the second handshake completed, so the lab did **not** claim that 0-RTT was successfully demonstrated.

Public Certificate Chain and TLS Decryption



Firefox SSLKEYLOGFILE allowed Wireshark to decrypt the TLS 1.3 session and expose the certificate chain.

Trust comparison

- ▶ Self-signed lab certificate:
Issuer = Subject.
- ▶ Public chain: server certificate signed by an Intermediate CA.
- ▶ Observed chain included WR2 and GTS Root R1.
- ▶ The browser trusts the chain because the root is already in the trusted certificate store.

TCP Reset Testing: From Failed Attempts to Success

The image shows a Wireshark packet capture of a network session. The filter bar at the top is set to 'tcp.port == 443'. The packet list shows a series of packets from 192.168.122.159 to 192.168.122.225. Packets 13 through 18 are highlighted in red, indicating a failed connection attempt. Packet 13 is a SYN packet (Seq=747777) and packet 14 is an ACK packet (Seq=747777). Packets 15 through 18 are RST packets (Seq=547777) generated by the client. Packet 19 is a successful SYN packet (Seq=547777) from the server. The packet details pane for packet 19 shows the following information:

No.	Time	Source	Destination	Protocol	Length	Info
13	9.552672510	192.168.122.159	192.168.122.225	TCP	74	777777 → 443 [SYN] Seq=747777
14	9.556584120	192.168.122.159	192.168.122.225	TCP	74	443 → 777777 [ACK] Seq=667777
15	9.557472965	192.168.122.159	192.168.122.225	TLSv1	402	Client Hello (SMI-den)
16	9.578088852	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777
17	9.580522622	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777
18	9.581232219	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777
19	9.581859090	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777
20	9.583206019	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777
21	9.583798586	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777
22	9.584487423	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777
23	9.584952972	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777
24	9.585516974	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777
25	9.586127736	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777
26	9.586724130	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777
27	9.587393270	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777
28	9.588138044	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777
29	9.588795502	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777
30	9.589495592	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777
31	9.589963684	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777
32	9.590637556	192.168.122.159	192.168.122.225	TCP	54	777777 → 443 [RST] Seq=547777

Session Actions Edit View Help

kali@PC1:~\$ sudo python3 rst_scapy.py
[sudo] password for kali:
Sending client-to-server RST: seq=595875053
RST sent

Next Sequence Number: 337 (relative seq)
Acknowledgment Number: 0
Acknowledgment number (raw): 0
0101 = Header Length: 20 bytes (5)
Flags: 0x004 (RST)
Window: 0192
[Calculated window size: 1048576]
[Window size scaling factor: 128]

A Scapy script generated RST packets, but invalid sequence numbers meant they were ignored.

Three approaches

- ▶ **hping3**: handshake completed too quickly to obtain a usable sequence number.
- ▶ **Scapy**: packets were sent, but the sequence context was still wrong.
- ▶ **tcppkill**: watched the live TCP session and generated valid RST packets from observed traffic.

Lesson

TCP reset injection depends on matching the active 4-tuple and valid sequence space; simply setting the RST flag is not enough.

HTTP/1.1 Reset: Transport Layer Still Matters

The image displays a Wireshark packet capture from the eth0 interface, filtered for 'tcp.port == 80'. The capture shows a sequence of events: a SYN exchange, a GET request, and a series of retransmissions. A red bracket highlights a period where the connection is reset. The terminal window below shows the execution of 'tcpkill' to inject a reset, followed by 'curl' attempts that result in 'Recv failure: Connection reset by peer'.

No.	Time	Source	Destination	Protocol	Length	Info
37	37.071189284	192.168.122.159	192.168.122.225	TCP	74	[TCP Retransmission]
38	38.099185172	192.168.122.159	192.168.122.225	TCP	74	[TCP Retransmission]
40	39.046309517	192.168.122.225	192.168.122.159	TCP	74	80 -> 46580 [SYN, ACK]
41	39.046388348	192.168.122.159	192.168.122.225	TCP	66	46580 -> 80 [ACK] Seq=
42	39.046589626	192.168.122.159	192.168.122.225	HTTP	145	GET / HTTP/1.1
43	40.075992922	192.168.122.225	192.168.122.159	TCP	74	[TCP Retransmission]
44	40.076047503	192.168.122.159	192.168.122.225	TCP	54	46580 -> 80 [RST] Seq=
46	41.102555170	192.168.122.225	192.168.122.159	TCP	74	[TCP Port numbers reu
47	41.102608618	192.168.122.159	192.168.122.225	TCP	54	46580 -> 80 [RST] Seq=
48	41.102832912	192.168.122.225	192.168.122.159	TCP	74	[TCP Port numbers reu
49	41.102847030	192.168.122.159	192.168.122.225	TCP	54	46580 -> 80 [RST] Seq=
50	42.050794954	192.168.122.225	192.168.122.159	TCP	66	[TCP ACKed unseen seq
51	42.050847139	192.168.122.159	192.168.122.225	TCP	54	46580 -> 80 [RST] Seq=
52	42.051050080	192.168.122.225	192.168.122.159	HTTP	919	HTTP/1.1 200 OK (tex
53	42.051061823	192.168.122.159	192.168.122.225	TCP	54	46580 -> 80 [RST] Seq=
54	42.383334539	192.168.122.159	192.168.122.225	TCP	54	46580 -> 80 [RST] Seq=

```
kali@PC1:~$ curl -i http://192.168.122.225/
* Trying 192.168.122.225:80 ...
* Connected to 192.168.122.225 (192.168.122.225) port
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 192.168.122.225
> User-Agent: curl/8.15.0
> Accept: */*
* Request completely sent off
* Recv failure: Connection reset by peer
* closing connection #0
curl: (56) Recv failure: Connection reset by peer

kali@PC1:~$ tcpkill -i eth0 host 192.168.122.159 and host 192.168.122.225 and port 80
tcpkill: listening on eth0 [host 192.168.122.159 and host 192.168.122.225 and port 80]
192.168.122.159:46580 > 192.168.122.225:80: R 1222534230:1222534230(0) win 0
192.168.122.159:46580 > 192.168.122.225:80: R 1222534732:1222534732(0) win 0
192.168.122.159:46580 > 192.168.122.225:80: R 1222535736:1222535736(0) win 0
192.168.122.159:46580 > 192.168.122.225:80: R 1222534230:1222534230(0) win 0
192.168.122.159:46580 > 192.168.122.225:80: R 1222534732:1222534732(0) win 0
```

tcpkill reset an HTTP/1.1 connection after a small server-side delay created enough time for injection.

Observed result

- ▶ Valid RST arrived in the active TCP sequence space.
- ▶ The request was interrupted before the HTTP exchange completed.
- ▶ curl reported Recv failure: Connection reset by peer.
- ▶ Encryption protects content, but it does not prevent a valid transport-layer reset from terminating a TCP session.

MITM Position and Manual RST Injection

```
kali@MITHM: ~  
Session Actions Edit View Help  
  
(kali@MITHM)-[~]  
$ sudo tshark -l -n -i eth0 \  
-o tcp.relative_sequence_numbers:false \  
-f "tcp and host 192.168.122.159 and host 192.168.122.225 and port 443" \  
-T fields \  
-e ip.src -e tcp.srcport -e ip.dst -e tcp.dstport -e tcp.seq -e tcp.ack -e tcp.flags  
Running as user "root" and group "root". This could be dangerous.  
Warning: program compiled against libxml 215 using older 214  
Capturing on 'eth0'  
  
192.168.122.159 6666 192.168.122.225 443 1930361340 0 0x0002  
192.168.122.159 6666 192.168.122.225 443 1930361340 0 0x0002  
192.168.122.159 6666 192.168.122.225 443 1930361340 0 0x0002  
192.168.122.159 6666 192.168.122.225 443 1930361340 0 0x0002  
192.168.122.159 6666 192.168.122.225 443 1930361340 0 0x0002  
192.168.122.159 6666 192.168.122.225 443 1930361341 1992904207 0x0000  
4  
192.168.122.159 6666 192.168.122.225 443 1930361340 0 0x0002  
192.168.122.159 6666 192.168.122.225 443 1930361340 0 0x0002  
192.168.122.159 6666 192.168.122.225 443 1930361341 1385756615 0x0000  
4  
192.168.122.159 6666 192.168.122.225 443 1930361341 1594904130 0x0000
```

After ARP spoofing, the MITM node could observe sequence and ACK values with tshark.

[illegible]

A fixed curl source port, server delay and observed sequence number allowed a spoofed RST to interrupt the TLS connection.

Timing matters

When an automated reset arrived too late, the TLS handshake completed and the reset only terminated the already-established session.

Security and Production Considerations

Certificate trust

Self-signed certificates are useful in labs, but production clients need a trusted chain or centrally managed private PKI. Repeatedly ignoring warnings trains unsafe behaviour and increases MITM risk.

0-RTT

QUIC 0-RTT can reduce latency for resumed sessions, but early data can be replayed. Use it only for operations that are safe to repeat.

Transport attacks

TLS encrypts and authenticates application data, but a valid TCP reset can still terminate a TCP-based TLS or HTTP session. Availability is a separate property from confidentiality.

Protocol selection

HTTP/2 keeps TCP and gains multiplexing; HTTP/3 moves to QUIC/UDP and integrates TLS 1.3, changing both latency and failure behaviour.

Project Summary

Implemented

- ▶ TLS 1.2 HTTPS service with a self-signed RSA certificate.
- ▶ Wireshark analysis of ClientHello, ServerHello, ECDHE, certificate and Application Data.
- ▶ HTTP/2 negotiation through ALPN under TLS 1.3.
- ▶ Caddy HTTP/3 service and QUIC handshake analysis.
- ▶ Public certificate-chain inspection using exported TLS session keys.
- ▶ TCP reset tests from direct and MITM positions.

Evidence and lessons

- ▶ X25519 ECDHE provided ephemeral key agreement in the observed sessions.
- ▶ QUIC integrated TLS 1.3 directly into the transport handshake.
- ▶ No clear 0-RTT application data was observed, so the result remained inconclusive.
- ▶ Successful RST injection required valid live TCP sequence information.

Security result

The lab connects cryptographic trust, protocol performance and transport-layer availability by validating both normal handshakes and deliberately interrupted sessions.